Time Series Forecasting of Air Quality Index

Kashyap Milind Trivedi[1], Akshat Kadia[1] and Krishil Jayswal[1]

[1]*Dhirubhai Ambani University*

Abstract

This study focuses on analyzing and forecasting air quality by examining time-series data, with a specific emphasis on the Air Quality Index (AQI). Utilizing a dataset comprising various pollutants such as PM2.5, PM10, NO2, NH3, SO2, CO, and OZONE, we begin by assessing the stationarity of the AQI time series using statistical methods like the Augmented Dickey-Fuller (ADF) test. Following this, classical time-series models ARIMA and SARIMA are implemented to model and predict AQI trends. These models are evaluated based on their accuracy and ability to capture the underlying patterns in the data. To enhance predictive performance and capture complex temporal dependencies, we also apply deep learning models including Long Short-Term Memory (LSTM), Recurrent Neural Networks (RNN), and Gated Recurrent Units (GRU). Additionally, Empirical Mode Decomposition (EMD) is employed to preprocess the AQI data and improve the performance of deep learning models. The performance of these neural network models is compared against traditional approaches to determine their effectiveness in air quality forecasting. This comprehensive analysis aims to contribute to environmental monitoring efforts and supports data-driven decision-making for pollution control and public health advisories.

Contents

202201191@daiict.ac.in (K. M. Trivedi); 202203029@daiict.ac.in (A. Kadia); 202203040@daiict.ac.in (K. Jayswal)

1. Problem Statement

Air pollution poses a significant threat to public health and environmental sustainability, especially in rapidly urbanizing regions. Accurate forecasting of the Air Quality Index (AQI) is essential for timely interventions and effective pollution management. However, predicting AQI is challenging due to the complex, non-linear, and time-dependent nature of environmental data. Traditional statistical models often fall short in capturing these intricate patterns, while advanced deep learning models require careful tuning and large datasets to perform effectively. This study aims to develop and compare both classical time-series models (ARIMA, SARIMA, SARIMAX) and deep learning approaches (LSTM, RNN, GRU) to forecast AQI using historical air quality data. To enhance the performance of deep learning models, Empirical Mode Decomposition (EMD) is applied to preprocess the AQI time series data. The goal is to identify the most effective method for short-term AQI prediction, thereby supporting proactive environmental decision-making and public health safety measures.

2. Data Collection

To build a reliable AQI (Air Quality Index) forecasting model, we first collected historical AQI data. The data was sourced from the official government website: https://airquality.cpcb.gov.in/AQI_India/ .

We developed a custom data scraping script in **TypeScript** to automate the collection process. The script interacted with the website's backend APIs, which returned the data in **JSON** format. This approach allowed us to systematically gather AQI readings across different time periods of **Gandhinagar**.

After scraping, the raw JSON data required additional processing. We used **Python** to clean, transform, and organize the data. Key steps included:

- Parsing the JSON structure
- Handling missing or inconsistent values
- Standardizing date-time formats
- Structuring the data into a tabular format suitable for analysis

The final output was a comprehensive **CSV** file containing time-series AQI records, which served as the input for model development and training.

3. About the Data

This study utilizes daily air quality data spanning from May 1, 2019, to February 28, 2025, specifically collected for the Gandhinagar Sector 10 region. The dataset provides valuable insights into the concentration levels of several key pollutants, including:

- PM2.5: Fine particulate matter with a diameter of 2.5 micrometers or smaller.
- PM10: Particulate matter with a diameter of 10 micrometers or smaller.
- NO2: Nitrogen dioxide, a key indicator of vehicular and industrial emissions.
- NH3: Ammonia, typically emitted from agricultural and industrial sources.
- SO2: Sulfur dioxide, primarily released by industrial processes and combustion.
- CO: Carbon monoxide, a toxic gas produced by incomplete combustion of fuels.
- O3: Ozone, which is both a pollutant at ground level and essential in the upper atmosphere.

4. Data Analysis

4.1. Data visualization

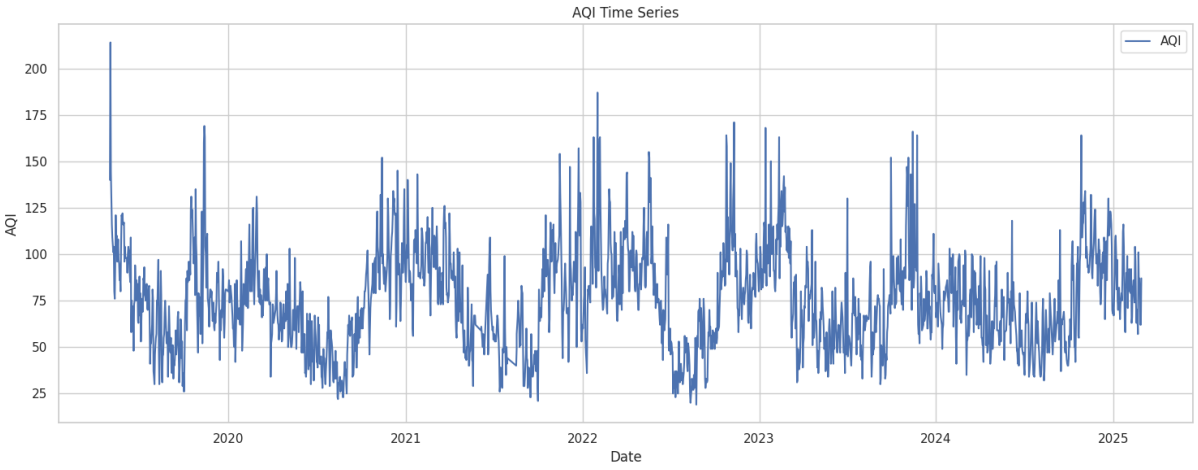

Figure 1: AQI daily data from 2019 to 2025

4.2. Data Decomposition

Any time series consists of three components trend, seasonality, and residual.

4.2.1. Choice of Decomposition Model

There are two primary types of time series decomposition:

1. **Additive Decomposition**

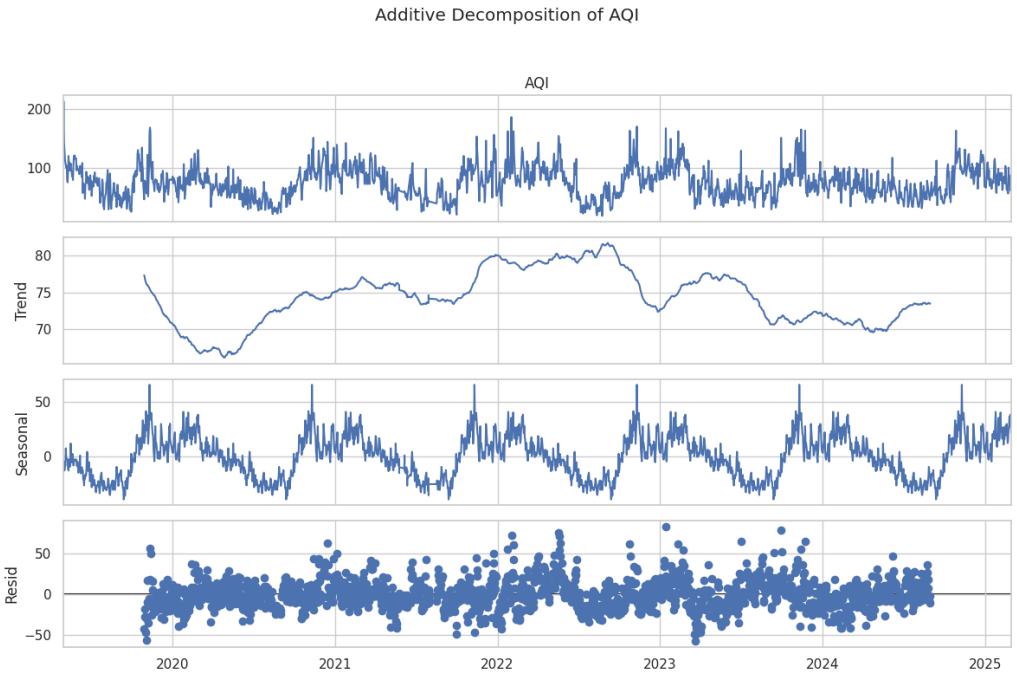

Figure 2: Additive decomposition

Additive Decomposition is a time series modeling technique where the series is expressed as the sum of three components:

$$Y_t = T_t + S_t + R_t$$

where Y_t is the observed value, T_t is the trend, S_t is the seasonal component, and R_t is the random (or residual) component.[1]

From this, we can say that there is some seasonality, mostly yearly seasonality exists.

2. **Multiplicative Decomposition Multiplicative decomposition** models a time series as the

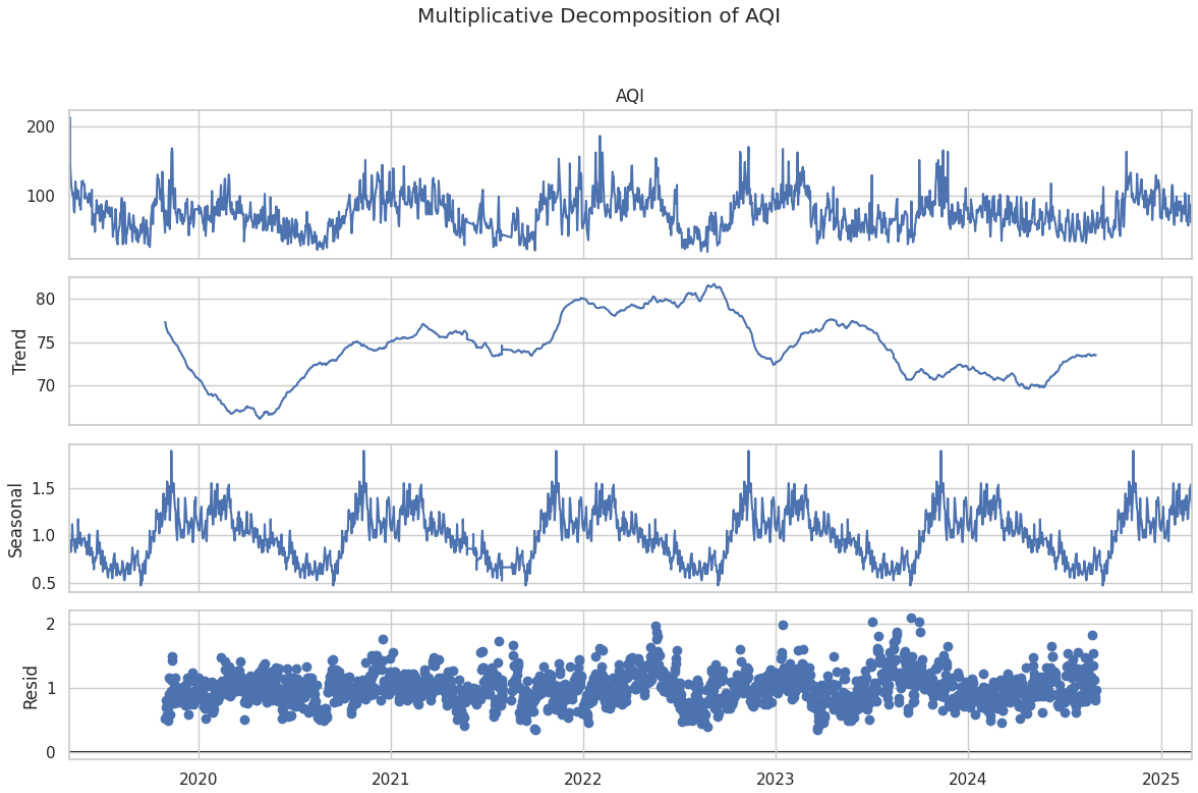

Figure 3: Multiplicative decomposition

product of its components:

$$Y_t = T_t \times S_t \times R_t$$

where Y_t is the observed value, T_t is the trend, S_t is the seasonal component, and R_t is the random (or residual) component.

4.3. Mean and Variance

4.3.1. Statistical Checks For Stationarity (ADF Test)

To test for stationarity, we ran the Augmented Dickey–Fuller test[1]; results are in Table [2].

Metric	Value
ADF Test Statistic	-5.044799
p-value	0.000018

The p-value is less than 0.05, but when we plot it rolling variance, its mean and variance vary with time (refer fig 4), which contradicts the result.

[1] `statsmodels.tsa.stattools.adfuller`

4.3.2. Plotting Rolling Mean and Variance

To check stationarity, we have taken the Rolling mean and rolling variance with **window size 30**. Figure 4 shows the mean and variance depend on time, so we can say that the series is **non-stationary**.

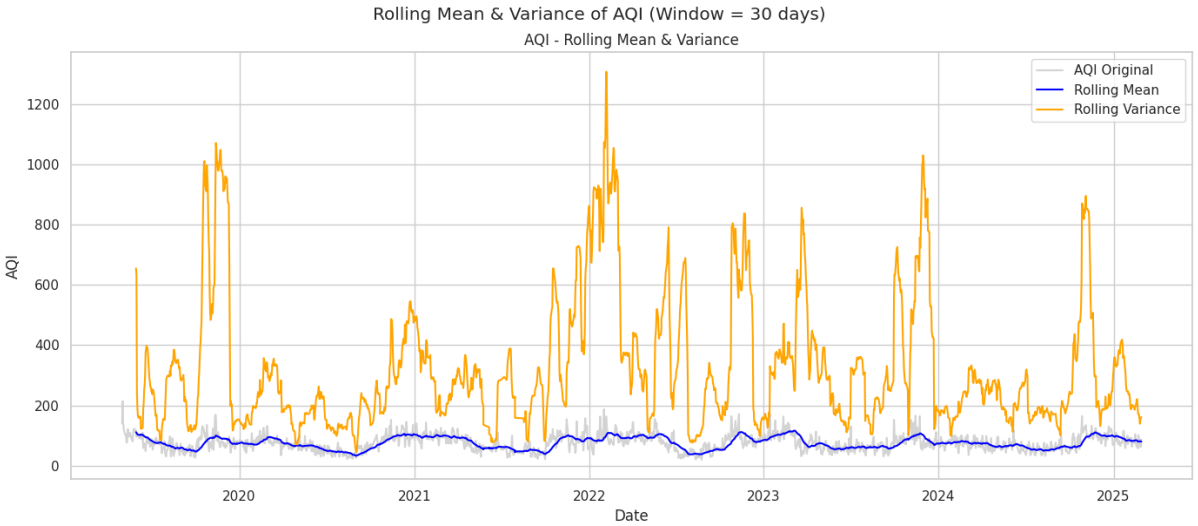

Figure 4: Rolling Mean and Variance

4.3.3. Transformations

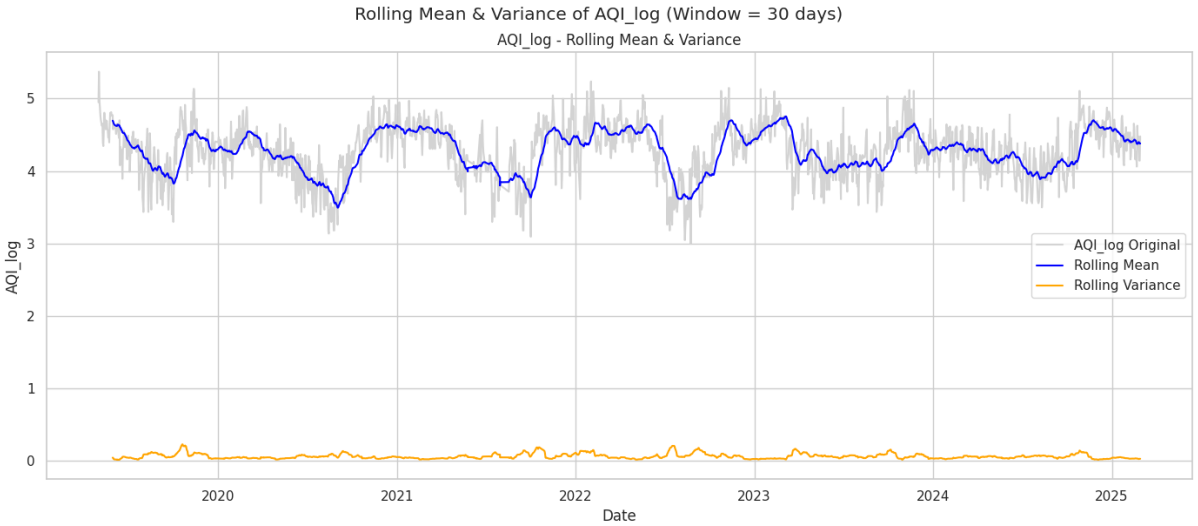

Figure 5: Log transformation to stabilize variance

Since, the time series is not stationary, we have apply **log transformation** to stabilize variance.

We have applied the first difference by using the df.diff() function in pandas. This stabilizes the variance, figure 6.

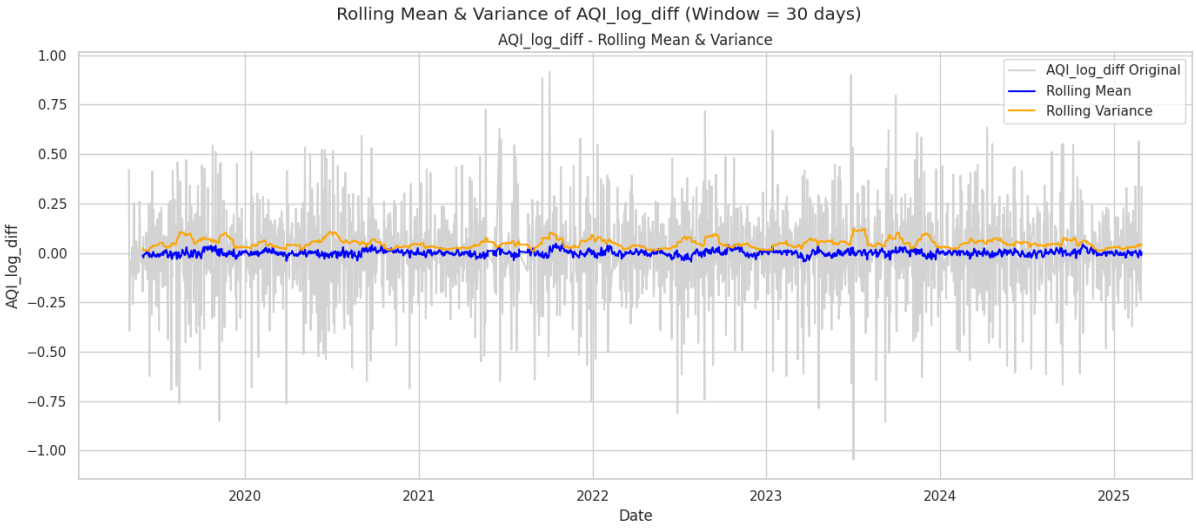

Figure 6: Differenced mean and Variance

4.4. ACF and PACF Plots

Partial Autocorrelation function and autocorrelation functions show how present values are dependent on past values. We can graphically predict the values of p and q of the Autoregressive Model AR(p) and Moving Average MA(q) using lags of the Partial Autocorrelation function and Autocorrelation function, respectively.

4.4.1. ACF and PACF plots for AQI

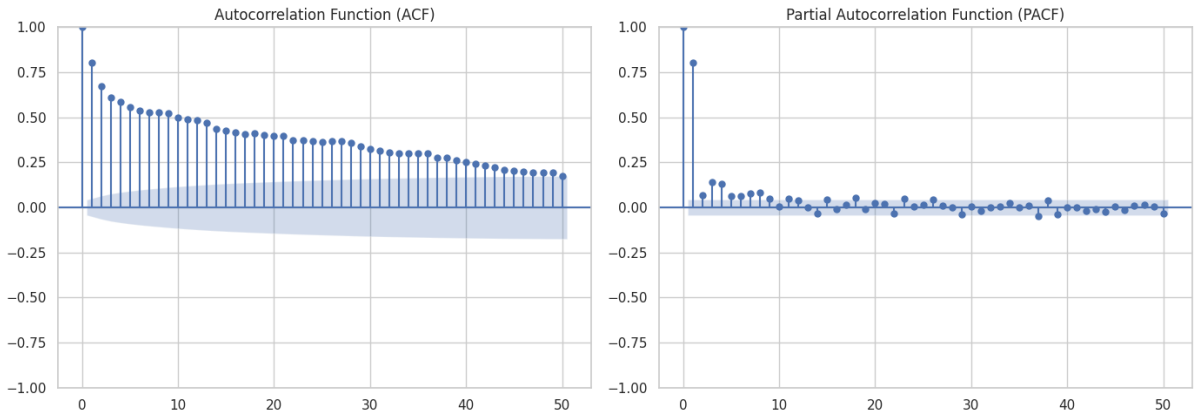

Figure 7: ACF and PACF of AQI

The ACF plot shows a gradual decay, indicating strong autocorrelation and suggesting non-stationarity in the time series. The PACF plot shows a sharp cutoff after lag 1, which suggests that an AR(1) model may be appropriate for modeling the data.

4.4.2. AQI Log ACF and PACF

The results are largely the same as AQI, so we can conclude that taking **log transformation** does not affect autocorrelation.

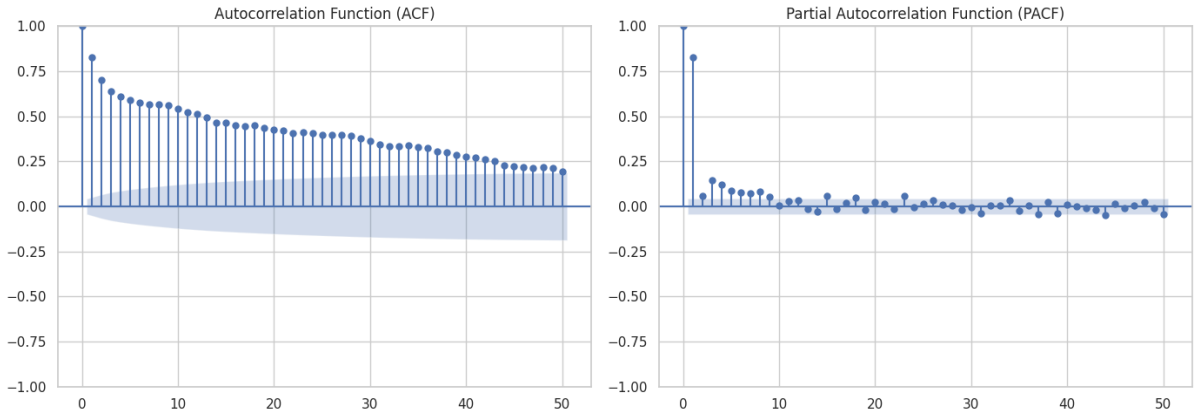

Figure 8: ACF and PACF of AQI Log

4.4.3. AQI Log differenced ACF and PACF

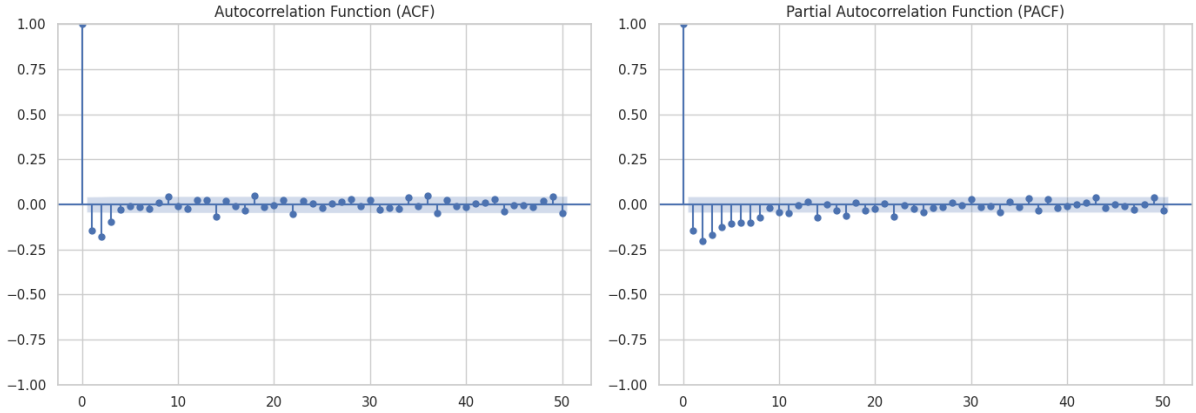

Figure 9: ACF and PACF of AQI Log differenced

After applying log transformation and first-order differencing, the ACF and PACF plots show no significant autocorrelations beyond lag 2 and 1, respectively, so we can conclude that it is stationary, and **ARMA(2,1)** and **ARIMA(2,1,1)** models will work.

5. Model Selection

5.1. Classical Methods

5.1.1. Historical Mean Method

This method uses the average of past data points to assign as future values.

RMSE: 22.96

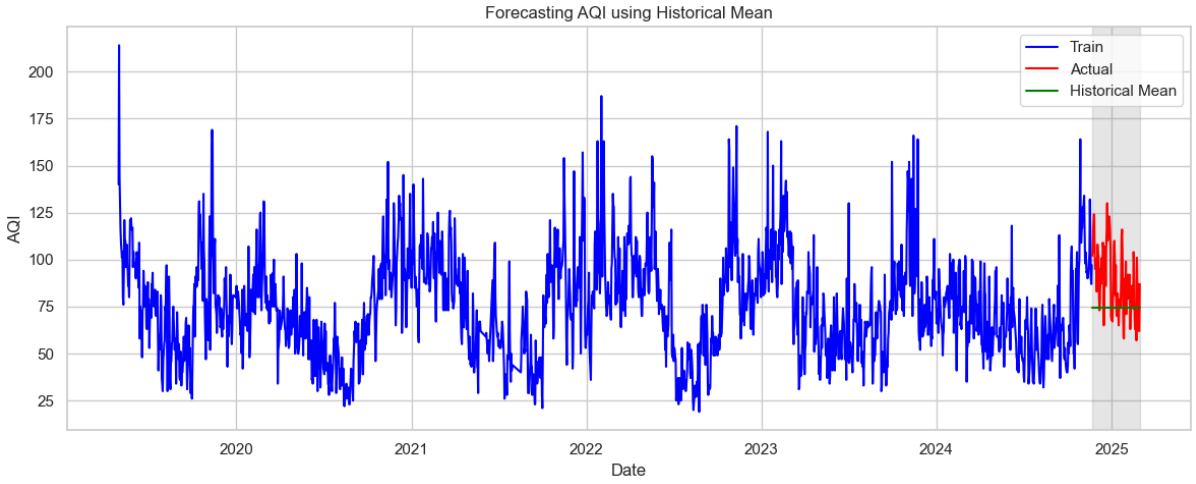

Figure 10: Historical Mean prediction

5.2. Naive Method

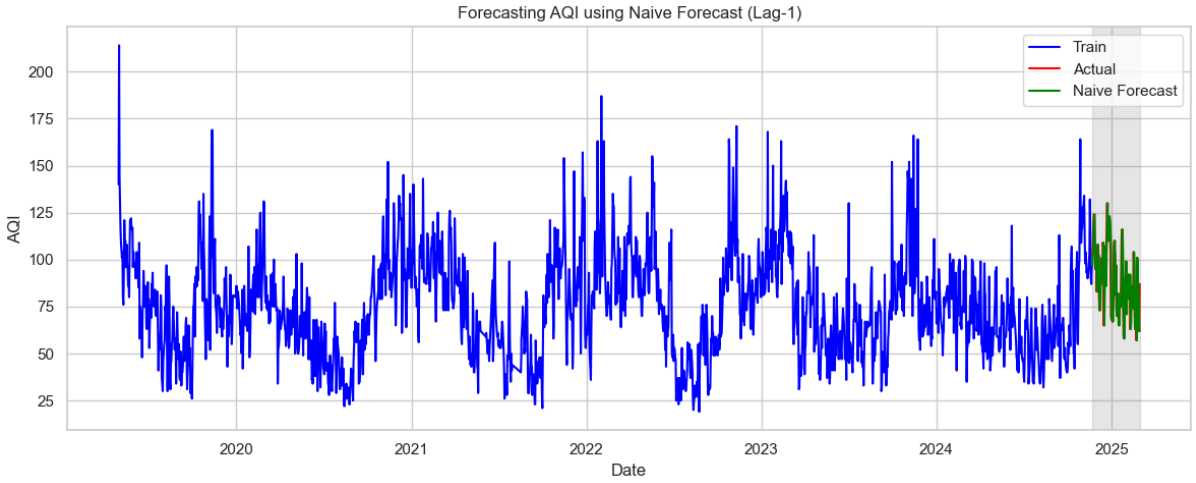

Figure 11: Naive Method

In this method, we forecast that the next period will be the same as the actual value of the previous period.

RMSE: 14.28

5.3. Drift Method

The drift method in forecasting is a simple approach where the forecast for a future time period is calculated by adding a constant drift (the average change in the historical data) to the last observed value

$$y_{(t+h)} = y_t + h * Drift$$

RMSE: 18.53

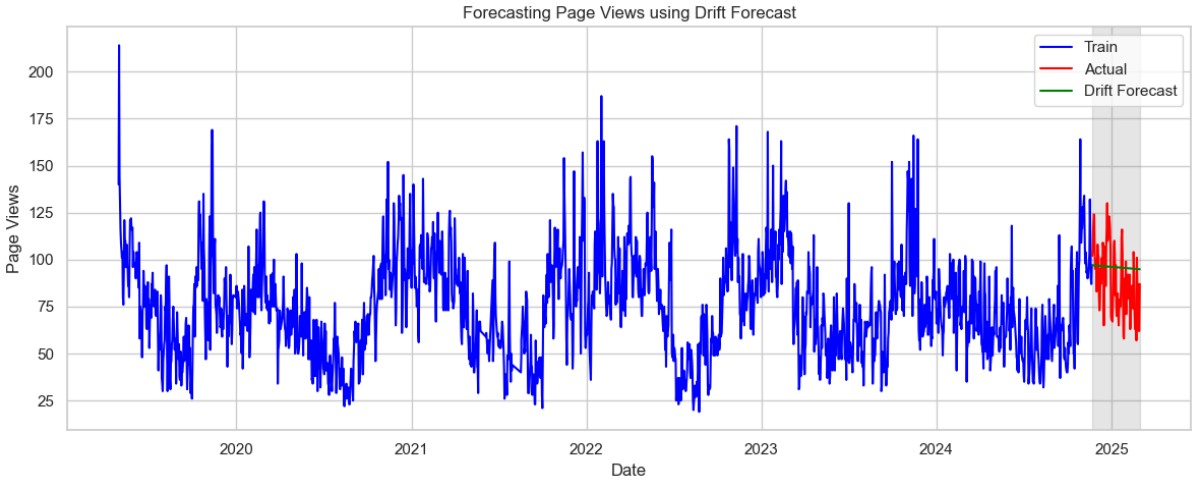

Figure 12: Drift Method

5.4. ARIMA

Since the AQI_log_diff series is stationary, it is suitable for model fitting. We consider an ARIMA(p,1,q).

Alternatively, the AR(p) part is some, and the differencing order is set to one, allowing the model to handle differencing internally. The MA component remains unchanged.

Using ARIMA(p,1,q) simplifies the process, as the model takes care of differencing automatically. However, since the predictions will be in the log-transformed scale, we need to invert them using the exponential function to get the forecasted AQI values on the original scale.

The ARIMA(p, d, q) model combines autoregression (AR), differencing (I), and moving average (MA) terms [3]. We performed grid search over $p \in [0, 5]$, $d = 1$, $q \in [0, 5]$ based on AIC and validation RMSE.

The ARIMA(2, 1, 1) model was evaluated on the scaled daily AQI dataset, and its performance was measured using the Root Mean Squared Error (RMSE) and Akaike Information Criterion (AIC).

$$\text{RMSE} = 4.28$$

This indicates a relatively low prediction error on the test dataset. Additionally, the AIC value for this configuration was -818.71, suggesting a good balance between model complexity and fit. The prediction plot clearly illustrates that the ARIMA(2, 1, 1) model captures the underlying trend and seasonal patterns present in the actual test data, closely aligning with the true values across the forecast horizon. This reinforces the model's capability to provide robust short-term forecasts for the given time series.

5.4.1. Diagnostic plots for ARIMA

- The top-left plot shows residuals over time, indicating no clear pattern and suggesting constant variance.
- The top-right plot is the autocorrelation function (ACF) of the residuals, showing little to no autocorrelation.
- The bottom-left Q-Q plot suggests that the residuals are approximately normally distributed, with minor deviations in the tails.
- The bottom-right density estimate is consistent with a bell-shaped curve centered around zero. These diagnostics collectively support the adequacy of the model.

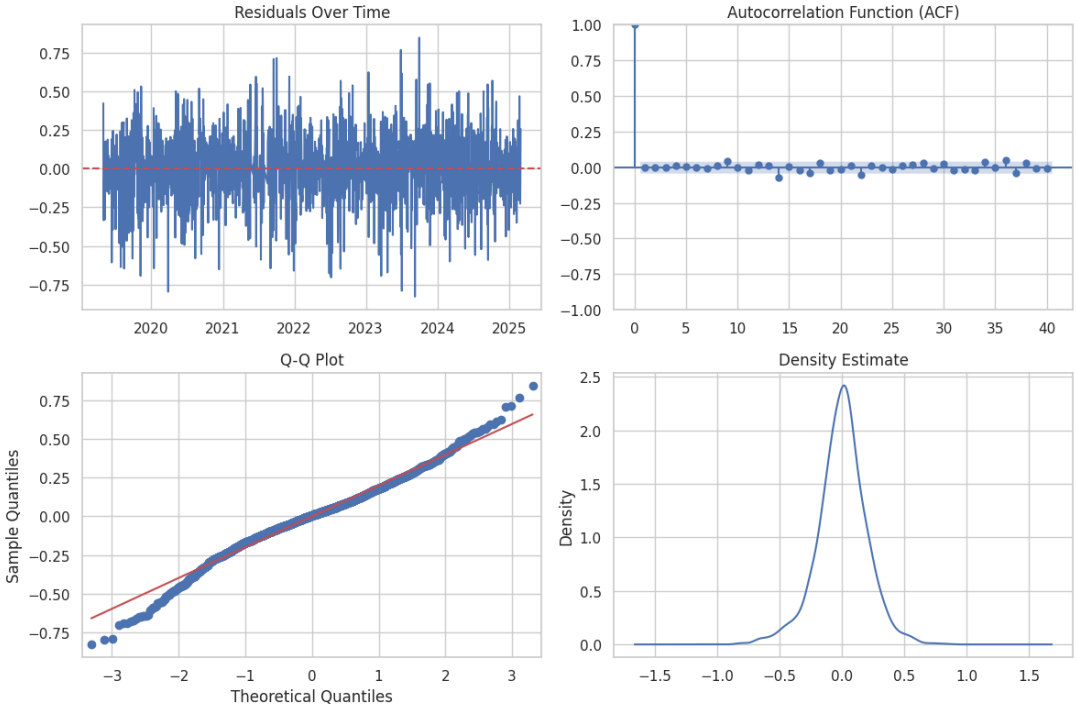

Figure 13: Diagnostic plots for ARIMA

5.4.2. Ljung-Box Test

The Ljung-Box test is a statistical test used to check for autocorrelation in a time series.

lb_stat	lb_pvalue
2.432398	0.991825

Here, p-value is > 0.05, so we are rejecting the hypothesis that is our residuals are not correlated. So cannot predict anything from this.

5.4.3. ARIMA Forecasting

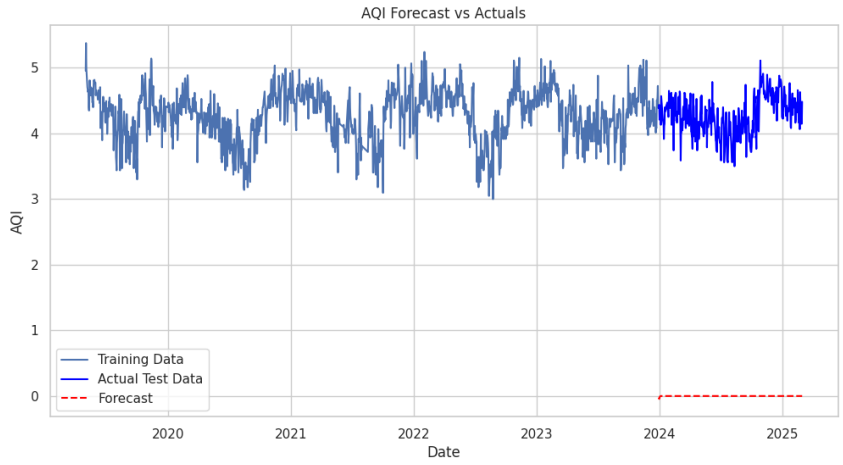

Figure 14: ARIMA Forecast

ARIMA Result:

- Figure 14 shows how well fit out forecast is.
- In the above figure we sees our model is not well-fitted.
- So this may be due to present seasonality in our data.

5.5. Find Seasonality/ SARIMA implementation

To model the temporal dependencies in the stationary differenced series, we employed the Seasonal ARIMA (SARIMA) framework, which extends the ARIMA model by incorporating seasonal autoregressive and moving average components. The data was split into training and test sets to validate forecasting performance. A grid search was performed over a range of SARIMA configurations, optimizing for the Akaike Information Criterion (AIC) to balance model fit and complexity.

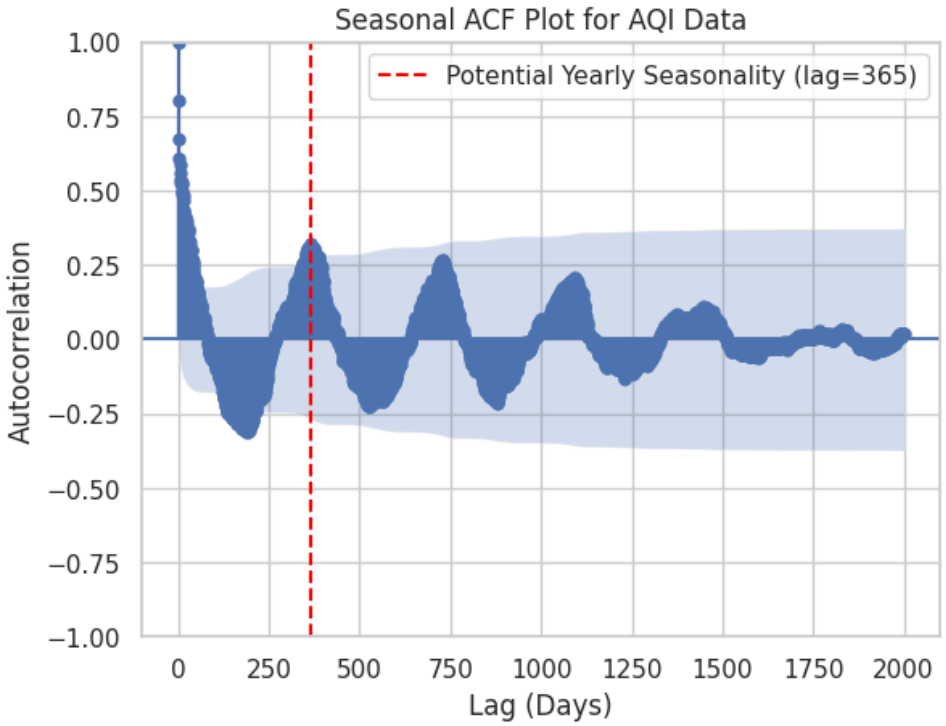

Figure 15: From this we can say that Our data had **yearly** seasonality. Because it has a bigger spike at 365,730,...

5.5.1. Seasonal Difference

After finding seasonality we apply seasonal differencing. Which is...

$$y_{t'} = y_t - y_{t-365}$$

After this we got the, our data looks like this.

Here, we can see some variation. So we apply a log transformation to it.

5.5.2. ARIMA on new data

After removing seasonality, we apply ARIMA again. and we get...

$$\text{RMSE} = 0.25$$

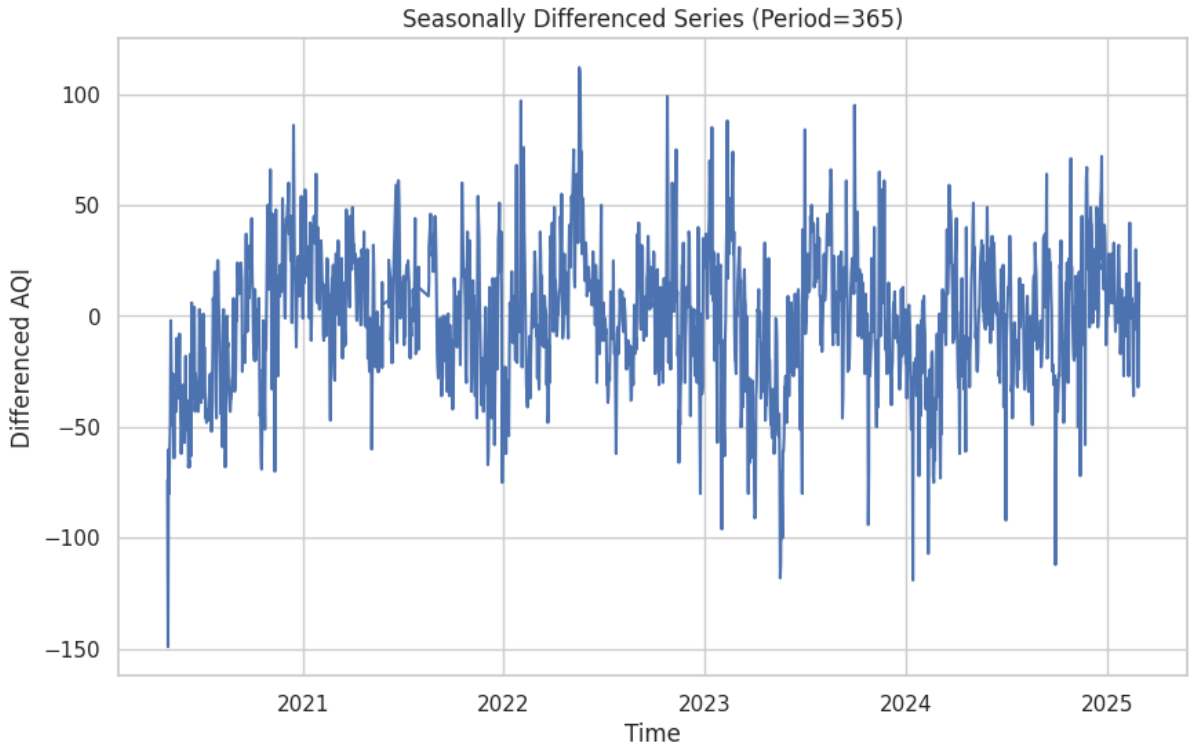

Figure 16: Plot after taking seasonal difference

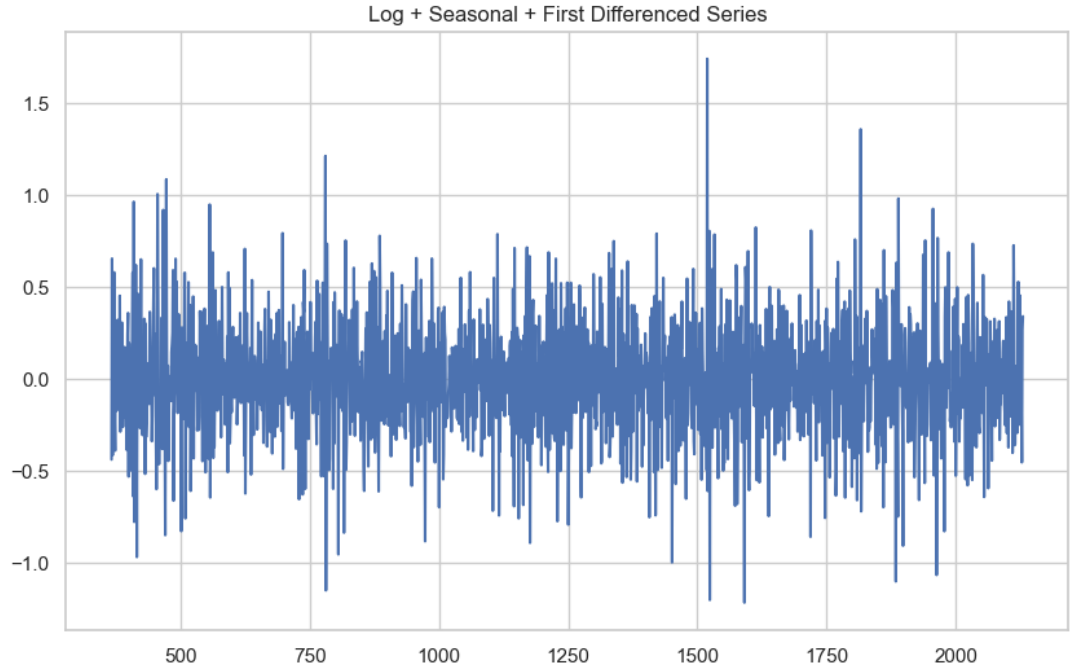

Figure 17: Log+seasonal difference

5.5.3. SARIMA

SARIMA is an extension of the ARIMA model that handles seasonal components in time series data. It's designed for data where patterns repeat at regular seasonal intervals. Our models parameters are,

$$SARIMA(2,1,1) \times (P,1,Q,365)$$

To find P, Q, we apply the same procedure that we use to find p,q.

Unfortunately, we are unable to apply SARIMA due to a lack of computational power.

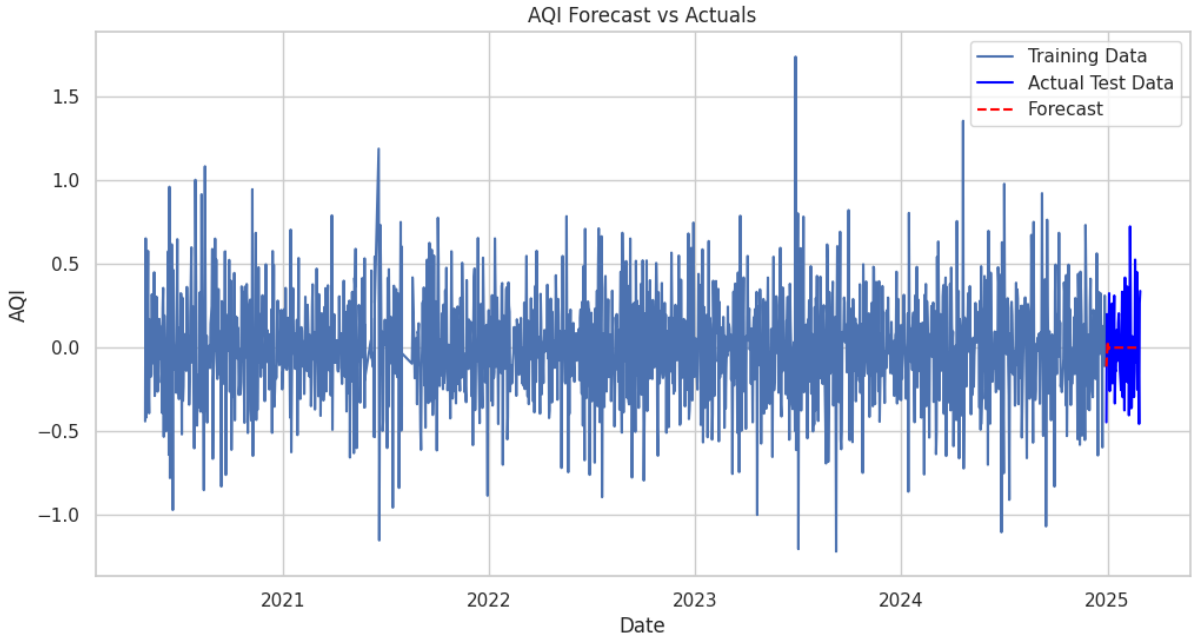

Figure 18: ARIMA after removing seasonality

6. Deep Learning Models

6.1. Introduction

This section presents an overview of Deep Learning Models such as Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Gated Recurrent Units (GRU). These models are particularly effective for sequential data and are widely used in areas like natural language processing and time-series prediction.

6.2. Recurrent Neural Network (RNN)

RNNs are one of the simplest types of recurrent architectures used to model sequential relationships in data. However, they often struggle to capture long-term dependencies in sequences due to issues like vanishing gradients.

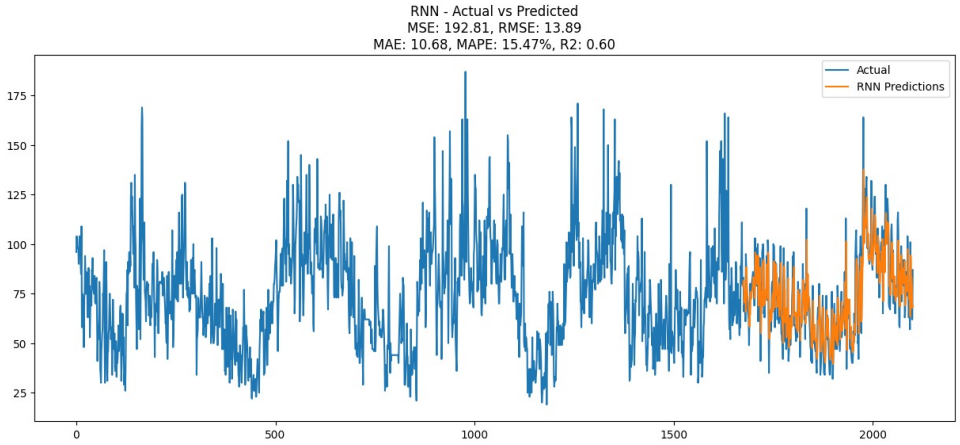

Figure 19: RNN Forecast

6.3. Long Short-Term Memory (LSTM)

LSTMs are a more advanced form of RNNs that include additional mechanisms called gates to regulate the flow of information. This makes them more capable of learning long-term dependencies and addressing the limitations of traditional RNNs.

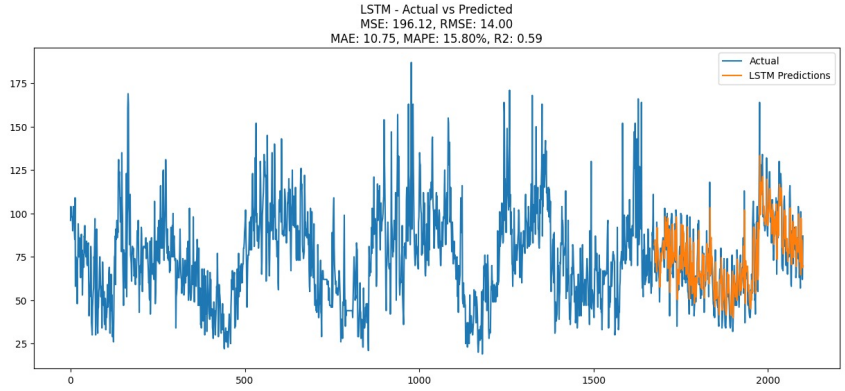

Figure 20: LSTM forecast

6.4. Gated Recurrent Unit (GRU)

GRUs are a simplified version of LSTMs that combine the forget and input gates into a single update gate. They are computationally more efficient while still maintaining the ability to model long-term dependencies in sequential data.

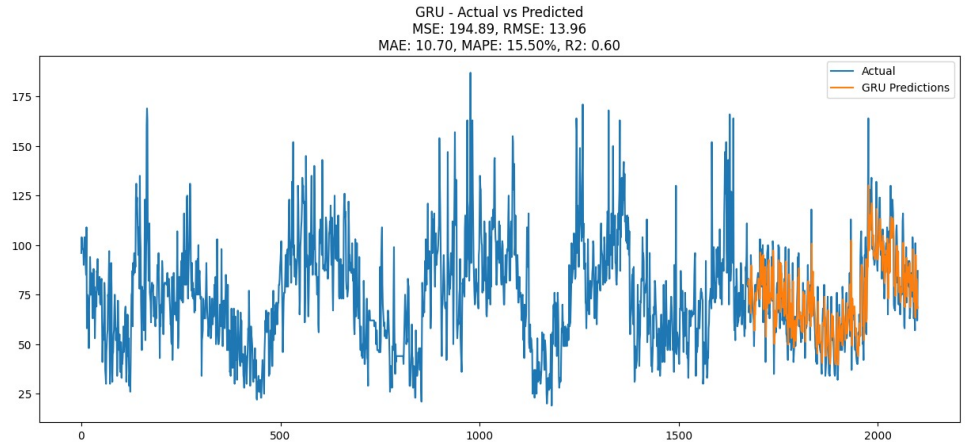

Figure 21: GRU forecast

6.5. Usage Summary

Note: For this study, we utilize **RNN**, **LSTM**, and **GRU** architectures to model the sequential data and evaluate their performance on the prediction task.

6.6. Comparison of Different Models

Model	Set	MSE	RMSE	MAE	R^2	MAPE
RNN	Training	233.3845	15.2769	11.0027	0.6807	16.1827
	Test	193.3429	13.9048	10.7085	0.5994	15.4890
GRU	Training	230.5107	15.1826	11.0221	0.6846	16.5973
	Test	196.1203	14.0043	10.8421	0.5937	16.0900
LSTM	Training	229.3199	15.1433	10.9745	0.6862	16.4648
	Test	196.7532	14.0269	10.7854	0.5924	15.9200

Table 1
Performance Comparison of Deep Learning Models

6.7. Forecast Plot

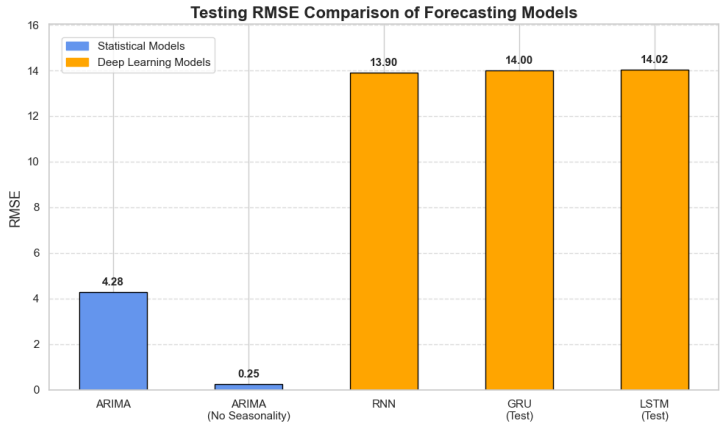

Figure 22: Forecast results using Deep Learning Models

7. Empirical Mode Decomposition on Deep Learning Models

Empirical Mode Decomposition (EMD) [4]is a method used to analyze non-linear and non-stationary signals—that means signals that change over time and don't have a fixed pattern, like heartbeats, stock prices, or audio signals.

It helps us break a complicated signal into simpler components called Intrinsic Mode Functions (IMFs). Each IMF shows a simple oscillation or wave-like pattern, kind of like breaking down music into different notes.

Why use EMD?
Traditional methods like the Fourier Transform assume signals are made of perfect sine waves. But many real-world signals aren't that neat. EMD doesn't make any assumptions—it learns directly from the data, making it data-driven and adaptive

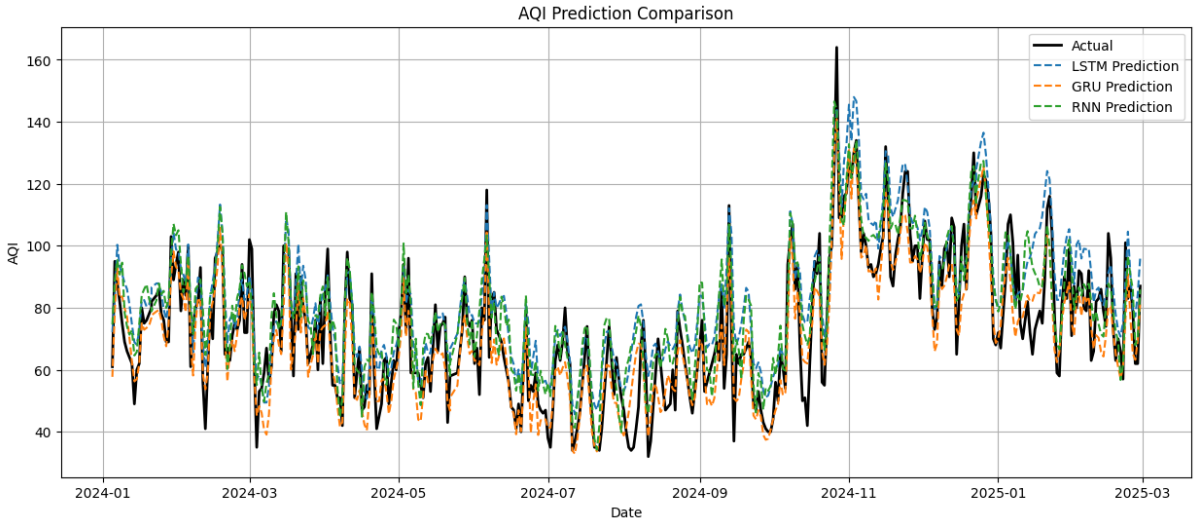

Figure 23: Forecasting after applying EMD

Table 2
Performance Metrics

Model	MAE	RMSE	R^2
LSTM	10.9235	13.3600	0.6350
GRU	7.5504	9.4723	0.8165
RNN	9.7138	11.8987	0.7105

8. Conclusion

RNNs and their variants, such as LSTMs and GRUs, are powerful tools for modeling sequences. While simple RNNs are limited by gradient issues, LSTM and GRU introduce gating mechanisms that allow for better performance and deeper architectures. To further enhance predictive accuracy, we applied Empirical Mode Decomposition (EMD) as a preprocessing step to decompose the time series into simpler, more structured components. This allowed the models to learn from denoised and mode-specific representations of the data. As a result, we observed a significant reduction in prediction error, with improvements in key performance metrics such as MAE, RMSE, and R^2 across all model architectures.

9. Key Learnings and Future Work

9.1. Key Learnings

- **Limitations of Simple RNNs:** Traditional RNNs suffer from vanishing and exploding gradient problems, limiting their ability to capture long-term dependencies in sequential data.
- **Advantages of LSTM and GRU:** LSTM and GRU architectures address gradient issues using gating mechanisms, which allow them to model longer sequences more effectively and support deeper neural networks.
- **Impact of Preprocessing with EMD:**
  - Empirical Mode Decomposition (EMD) was used as a preprocessing step to break down the time series into simpler Intrinsic Mode Functions (IMFs).
  - This helped to denoise the data and extract more structured and meaningful components.
- **Enhanced Predictive Accuracy:**
  - The models trained on EMD-processed data demonstrated significantly improved performance.
  - Metrics such as Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and the Coefficient of Determination (R^2) all showed substantial improvement across different model architectures.
- **Synergy Between Model Design and Preprocessing:** Combining advanced recurrent models with data decomposition techniques like EMD creates a powerful framework for accurate and robust time series prediction.

10. Limitations and Future Directions

Limitations

- **Computational Complexity of EMD:** Empirical Mode Decomposition is computationally expensive, particularly for large datasets, increasing overall model training and execution time.

- **Dependence on Data Characteristics:** The success of both EMD and deep learning models is highly dependent on the characteristics of the input time series, such as non-stationarity and noise, which can limit generalizability.
- **Hyperparameter Tuning:** Models like LSTMs and GRUs require careful tuning of hyperparameters, such as the number of layers and hidden units, which can be time-consuming and computationally demanding.

Future Directions

- **Use of Advanced Decomposition Methods:** Future work can explore more robust alternatives to EMD, such as Ensemble EMD (EEMD), Variational Mode Decomposition (VMD), or Wavelet Transforms, to enhance decomposition quality and address mode mixing issues.
- **Hybrid Model Architectures:** Combining LSTM/GRU models with other architectures like Convolutional Neural Networks (CNNs) or Transformers can help in capturing both spatial and temporal patterns more effectively.
- **Real-Time and Online Learning:** Developing models capable of real-time or online learning would be beneficial for dynamic, real-world applications requiring continuous updates.

References

[1] R. J. Hyndman, G. Athanasopoulos, Forecasting: principles and practice, OTexts, 2018.

[2] D. A. Dickey, W. A. Fuller, Distribution of the estimators for autoregressive time series with a unit root, Journal of the American statistical association 74 (1979) 427–431.

[3] G. E. Box, G. M. Jenkins, G. C. Reinsel, G. M. Ljung, Time Series Analysis: Forecasting and Control, John Wiley & Sons, 2015.

[4] N. E. Huang, Z. Shen, S. Long, M. Wu, H. Shih, Q. Zheng, N. Yen, C.-H. Tung, H. Liu, Empirical mode decomposition and the hilbert spectrum for nonlinear and non-stationary time series analysis, Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences 454 (1998) 903–995.